Zephyr Project

IT-Schulungen.com
Technology & Transformation

TCP als vorbereiteter Telemetrie-Transport

Adaptergrenze, GET und definierter Cleanup

4.11

Drei Grenzen – drei Verantwortungen

TRANSPORTSCHNITTSTELLE

Stabile Grenze

Start und Stop
Request und Antwort
Status zurückmelden
Transport austauschbar

TCP-MODUL

Mechanik kapseln

Session aufbauen
Bytes empfangen
vollständig senden
Cleanup ausführen

FACHLOGIK

Bleibt unabhängig

keine Deskriptoren
keine IP-Adressen
keine Framingdetails

Application State und Kommunikationskanal treffen sich nur an der gemeinsamen Transportgrenze.

Lebenszyklus des vorbereiteten TCP-Moduls

`L4 verfügbar`	Modul starten oder für den Verbindungsaufbau freigeben
`Session aufbauen`	Endpunkt verbinden und Empfangszustand initialisieren
`GET verarbeiten`	vollständigen Request erkennen und letzten Sensorstatus lesen
`Antwort senden`	Teilübertragungen fortsetzen, bis der vollständige Frame gesendet ist
`Cleanup`	Session genau einmal schließen und definierten Ausgangszustand herstellen

Bei verlorenem L4-Zustand wird der Adapter beendet – die Sensor-Pipeline läuft weiter.

Minimaler Vertrag: GET und eine Antwort

KLEINER PROTOKOLLVERTRAG

Die Demo braucht genau einen Request und ein eindeutig begrenztes Antwortformat

Request	GET mit Zeilenabschluss fordert die letzte gültige Probe an
Antwort	Sequenznummer, Alter, Qualität und Messwert plus Zeilenabschluss
Framing	vollständigen Frame erst nach erkanntem Abschluss verarbeiten
Restdaten	unverbrauchte Bytes für den nächsten Request im Puffer behalten

Der Vertrag bleibt absichtlich klein; Protokolloptimierung ist nicht Ziel dieses Blocks.

TCP-Bytestrom kompakt verarbeiten

AKKUMULATIONSPFAD

```
recv → append → scan → extract → respond
```

Empfangen	Bytebereich und tatsächliche Länge übernehmen
Akkumulieren	nur innerhalb der begrenzten freien Pufferkapazität anhängen
Frame erkennen	GET erst nach vollständigem Zeilenabschluss ausführen
Rest erhalten	unverbrauchte Bytes im Puffer nach vorn verschieben
Antworten	Antwort vollständig senden und Teilübertragungen berücksichtigen

Aufrufgrenzen von recv() und send() sind keine Grenzen des Anwendungsprotokolls.

Drei Empfangsergebnisse – drei Reaktionen

01	`recv() == 0`	Peer hat geordnet geschlossen; Restframe verwerfen und Session bereinigen
02	`Timeout`	Wartezeit ist abgelaufen; Zustand protokollieren und definierten Fehlerpfad wählen
03	`Negativer Fehlerwert`	konkreten Fehler klassifizieren; kein automatisches endloses Retry

Alle drei Pfade enden in einem bekannten Sessionzustand – niemals in halbem Cleanup.

Fünf Pflichten des TCP-Adapters

Kleine Mechanik, aber eindeutig definierte Zustands- und Ressourcenübergänge

01
L4-Gate
nur bei Konnektivität starten

02
Teil-Send
bis zum Frameende fortsetzen

03
Fehler trennen
Close, Timeout und errno

04
Genau einmal
Session-Ressource schließen

05
Zurücksetzen
Puffer und Zustand definieren

Der Adapter kehrt nach jeder Session in denselben überprüfbaren Ausgangszustand zurück.

Transportgrenze bleibt austauschbar

GEMEINSAME SCHNITTSTELLE

```
transport_start()
transport_request()
transport_stop()

Input: Application State
Output: Status + Daten

keine Socketdetails außen
```

01 TCP unter native_sim

verbindlicher Demonstrationspfad mit vorbereiteten Endpunkten und Hostclient.

02 Weitere Transporte

BLE oder Devkit-spezifische Kanäle implementieren später dieselbe Grenze.

Der Transport kann wechseln, ohne Sensorerfassung oder Qualitätsmodell zu verändern.

Lernkontrolle: TCP-Minimalvertrag

ERWARTET

```
L4 gate ........ PASS
GET frame ...... PASS
remainder ...... PASS
partial send ... PASS
cleanup once ... PASS
```

01 Grenze zeigen

Application State bleibt frei von Socket-Deskriptoren und Framingdetails.

02 Pflichten benennen

Bytestrom, Teilübertragung, Peer-Close, Timeout, Fehler und Cleanup erklären.

Ergebnis: Der vorbereitete TCP-Adapter ist klein, testbar und als Transport austauschbar.